Module 5

Advanced UVM Concepts

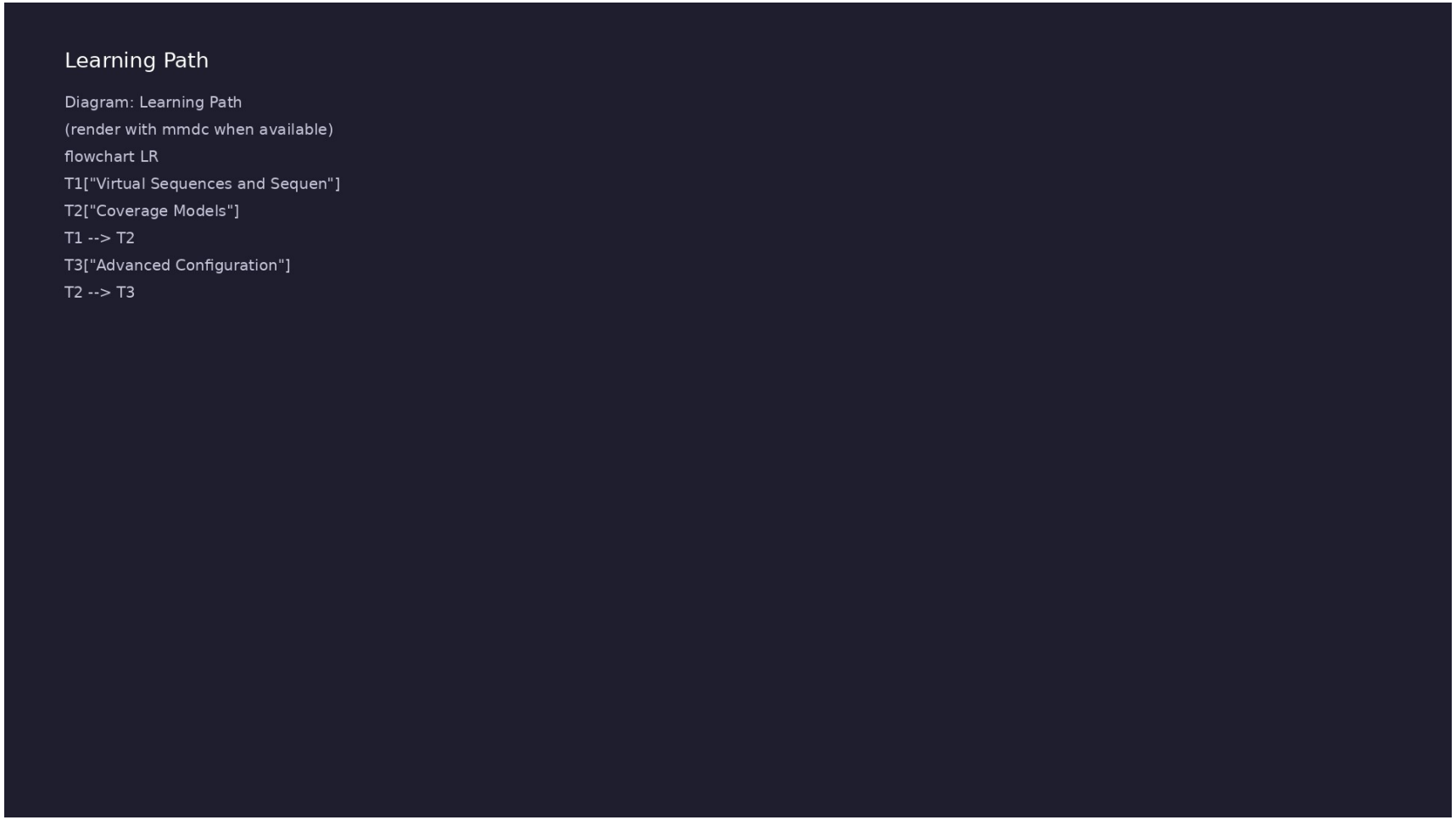
Learning objectives

- Master advanced UVM features and patterns

Prerequisites

- See module README

Learning path



Master advanced UVM features and patterns

Overview

- This module covers advanced UVM concepts including virtual sequences, coverage models, advanced con...

Syllabus topics

1. Virtual Sequences and Sequencers (1/9)

- Virtual Sequence Purpose
- Coordinating multiple sequencers
- Cross-agent stimulus generation
- System-level sequence coordination
- Sequence libraries
- Virtual Sequencer Implementation

1. Virtual Sequences and Sequencers (2/9)

- Virtual sequencer structure
- Sequencer references
- Connection in connect_phase
- Virtual sequencer usage
- VirtualSequencer: Contains references to multiple sequencers (`master_seqr`, `slave_seqr`)
- VirtualSequence: Coordinates sequences on multiple sequencers

1. Virtual Sequences and Sequencers (3/9)

- ChannelSequence: Regular sequence for a single channel
- Parallel Execution: Using `fork-join` for concurrent sequences
- Sequential Execution: Sequential sequence coordination
- Purpose: Hold references to multiple sequencers
- Key: References are set in `connect_phase()` from environment
- Note: Virtual sequencer doesn't create sequencers, only references them

1. Virtual Sequences and Sequencers (4/9)

- Purpose: Coordinate sequences on multiple sequencers
- Key: Virtual sequence doesn't extend `uvm_sequence #(transaction_type)`
- Parallel: Use `fork-join` for concurrent execution
- Sequential: Call `start()` sequentially for ordered execution
- Purpose: Connect virtual sequencer/sequence to actual sequencers
- Usage: In test's `connect_phase()`

1. Virtual Sequences and Sequencers (5/9)

- Key: Must be set before starting virtual sequence
- Purpose: Start virtual sequence execution
- Key: Virtual sequence can start on virtual sequencer or null
- Purpose: Execute sequences concurrently
- Key: Both sequences run in parallel, `join` waits for both
- Timing: Total time = max(seq1_time, seq2_time)

1. Virtual Sequences and Sequencers (6/9)

- Use case: Independent operations, concurrent testing
- fork-join: Wait for all sequences (most common)
- fork-join_any: Wait for first to complete
- fork-join_none: Fire and forget
- Purpose: Execute sequences in order
- Key: Second sequence starts after first completes

1. Virtual Sequences and Sequencers (7/9)

- Timing: Total time = seq1_time + seq2_time
- Use case: Ordered operations, dependencies
- Purpose: Check sequencer availability
- Key: Always check for null before using sequencer references
- Virtual sequencer contains references to multiple sequencers
- Virtual sequence coordinates sequences on different sequencers

1. Virtual Sequences and Sequencers (8/9)

- Parallel sequence execution using fork-join enables concurrent testing
- Sequential sequence execution enables ordered coordination
- Virtual sequencer - Holds references, doesn't create sequencers
- Virtual sequence - Coordinates sequences on multiple sequencers
- Sequencer references - Set in `connect_phase()` from environment
- Parallel execution - `fork-join` for concurrent sequences

1. Virtual Sequences and Sequencers (9/9)

- Sequential execution - Sequential `start()` calls
  for ordered execution
- Virtual sequencer creation and connection
- Virtual sequence coordination
- Parallel sequence execution
- Sequential sequence execution

2. Coverage Models (1/8)

- Functional Coverage
- Coverage model structure
- Coverage sampling
- Coverage analysis
- Coverage reporting
- Coverage Types

2. Coverage Models (2/8)

- Data coverage
- Address range coverage
- Command coverage
- Cross coverage
- CoverageModel: Extends `uvm_subscriber` for coverage collection
- Coverage Sampling: Via `write()` method from analysis port

2. Coverage Models (3/8)

- Coverage Types: Data, address range, command, and cross coverage
- Coverage Analysis: Coverage statistics and reporting
- Purpose: Sample coverage data from transactions
- Usage: Called automatically when monitor writes to analysis port
- Key: Uses associative arrays to track unique values
- Purpose: Track unique values and counts

2. Coverage Models (4/8)

- Key: Associative arrays enable efficient coverage tracking
- Purpose: Report coverage statistics
- Usage: Called in cleanup phase
- Key: Uses `num()` to count unique keys
- Purpose: Connect producer to coverage model
- Key: Coverage automatically receives via `write()` method

2. Coverage Models (5/8)

- Purpose: Track combinations of multiple fields
- Key: 2D associative arrays for cross coverage
- Usage: Track (data, command) pairs, (address, data) pairs, etc.
- Purpose: Initialize coverage data structures
- Key: Fixed arrays initialized in constructor
- Coverage model class extending `uvm_subscriber` enables automatic sampling

2. Coverage Models (6/8)

- Coverage sampling via `write()` method receives transactions
- Coverpoints and bins track unique values and ranges
- Cross coverage between multiple fields tracks combinations
- `write(t)` - Receive transaction for coverage sampling
- Associative arrays - Track unique values efficiently
- `exists(key)` - Check if key exists in associative array

2. Coverage Models (7/8)

- `num()` - Count number of unique keys
- `foreach` - Iterate over associative array keys
- 2D associative arrays - Track cross coverage pairs
- Coverage sampling demonstrated
- Data coverage tracking
- Address range coverage

2. Coverage Models (8/8)

- Command coverage tracking
- Cross coverage analysis

3. Advanced Configuration (1/7)

- Complex Configuration Objects
- Configuration object design
- Configuration fields
- Configuration methods
- Configuration copying
- Configuration Hierarchy

3. Advanced Configuration (2/7)

- Environment-level configuration
- Agent-level configuration
- Component-specific configuration
- Configuration inheritance
- AgentConfig: Complex configuration object with multiple fields
- EnvConfig: Environment-level configuration containing agent configs

3. Advanced Configuration (3/7)

- Configuration Hierarchy: Multi-level configuration structure
- Configuration Usage: Setting and getting complex configuration
- Purpose: Enable configuration copying
- Usage: Allows configuration inheritance and copying
- Key: Must copy all configuration fields
- Purpose: Multi-level configuration hierarchy

3. Advanced Configuration (4/7)

- Key: Configuration objects can contain other configuration objects
- Purpose: Set configuration for specific components
- Key: Use component path in `set()` call
- Purpose: Retrieve configuration in component
- Key: Always check return value, provide defaults
- Purpose: Debug configuration values

3. Advanced Configuration (5/7)

- Usage: Logging, debugging
- Complex configuration objects enable rich configuration data
- Configuration hierarchy supports multi-level configuration
- Resource database usage provides flexible configuration access
- Configuration inheritance enables configuration reuse
- `do_copy()` - Enable configuration copying

3. Advanced Configuration (6/7)

- Nested configs - Configuration objects containing other configs
- Component-specific paths - Set config for specific components
- Config hierarchy - Multi-level configuration structure
- `convert2string()` - Debug configuration values
- Complex configuration object creation
- Configuration hierarchy demonstrated

3. Advanced Configuration (7/7)

- Configuration setting and getting
- Configuration object usage

4. Callbacks (1/8)

- Callback Mechanism
- Callback base classes
- Callback registration
- Pre/post callbacks
- Callback execution
- Callback Usage Patterns

4. Callbacks (2/8)

- Driver callbacks
- Monitor callbacks
- Transaction modification
- Side-effect actions
- DriverCallbackBase: Base callback class for driver
- DriverCallback: Driver callback implementation

4. Callbacks (3/8)

- MonitorCallbackBase: Base callback class for monitor
- MonitorCallback: Monitor callback implementation
- Callback Registration: Registering callbacks with components
- Purpose: Define callback interface
- Key: Virtual functions to be overridden in derived classes
- Purpose: Implement callback behavior

4. Callbacks (4/8)

- Key: `uvm_register_cb` macro registers callback type
- Purpose: Register callback type with component type
- Usage: Must be in callback class definition
- Key: Enables type-safe callback registration
- Purpose: Execute all registered callbacks
- Usage: Call at appropriate points in component code

4. Callbacks (5/8)

- Key: Executes all callbacks registered for this component instance
- Purpose: Connect callback to component instance
- Usage: In test's `connect_phase()`
- Key: Callback must be registered before it can be executed
- Purpose: Manage callbacks
- Key: Static methods operate on callback registry

4. Callbacks (6/8)

- Purpose: Multiple callbacks for same component
- Key: All registered callbacks execute in registration order
- Purpose: Control callback execution order
- Key: Pre-callbacks → operation → post-callbacks
- Callback mechanism enables non-intrusive extension
- Pre/post callbacks provide hooks for customization

4. Callbacks (7/8)

- Callback registration connects callbacks to components
- Callback usage patterns enable flexible test customization
- `uvm_register_cb` - Register callback type
- `uvm_do_callbacks` - Execute all registered callbacks
- Callback base class - Define callback interface
- Callback registration - Connect callback to component

4. Callbacks (8/8)

- Pre/post callbacks - Hooks before/after operations
- Callback registration demonstrated
- Pre-drive callbacks executed
- Post-drive callbacks executed
- Monitor callbacks executed

5. Register Model (1/9)

- Register Model Basics
- Register model structure
- Register blocks, registers, fields
- Register read/write operations
- Register predictor
- Register Model Components

5. Register Model (2/9)

- RegisterField: Individual register fields
- Register: Complete registers
- RegisterBlock: Register address spaces
- Register operations
- RegisterField: Represents a register field with offset, width, and value
- Register: Represents a register with address and fields

5. Register Model (3/9)

- RegisterBlock: Represents a register block with multiple registers
- Register Operations: Read/write operations on registers
- Purpose: Represent individual register fields
- Key: Fields have offset, width, and value
- Purpose: Add field to register
- Usage: Build register structure

5. Register Model (4/9)

- Key: Fields stored in queue
- Purpose: Write value to register
- Key: Updates register value and all field values
- Field extraction: Uses bit shifting and masking
- Purpose: Read register value
- Returns: Register value

5. Register Model (5/9)

- Purpose: Add register to block
- Key: Automatically assigns address based on position
- Purpose: Look up register by name
- Key: Returns null if register not found
- Purpose: High-level register access
- Usage: Access registers by name

5. Register Model (6/9)

- Key: Handles null checks
- Purpose: Extract field values from register value
- Key: Bit shifting and masking for field extraction
- Example: Field at offset 4, width 3: `(data >> 4) & 7`
- Purpose: Compose register value from field values
- Key: Shift and OR field values into register

5. Register Model (7/9)

- Purpose: Automatically assign register addresses
- Key: Address = base_address + (index * 4) for 32-bit registers
- Register model structure provides abstraction for register access
- Register blocks, registers, fields organize register hierarchy
- Register read/write operations enable register verification
- Simplified register model demonstrates concepts (full implementation requires uvm_reg)

5. Register Model (8/9)

- `add_field()` - Add field to register
- `write(data)` - Write value to register
- `read()` - Read value from register
- `add_register()` - Add register to block
- `get_register(name)` - Look up register by name
- `write_register()` - High-level write by name

5. Register Model (9/9)

- `read_register()` - High-level read by name
- Field extraction - Bit shifting and masking for field values
- Register model structure demonstrated
- Register field operations
- Register read/write operations
- Register block management

6. Advanced UVM Methods Reference (1/5)

- Sequencer References: Hold references to multiple sequencers
- Connect Phase: Set references from environment
- No Transaction Type: Virtual sequencer doesn't extend `uvm_sequencer #(type)`
- Body Task: Coordinates sequences on multiple sequencers
- Sequencer References: Set before starting sequence
- Parallel Execution: `fork-join` for concurrent sequences

6. Advanced UVM Methods Reference (2/5)

- Sequential Execution: Sequential `start()` calls
- `write(t)` - Receive transaction for sampling
- Associative Arrays - Track unique values
- `exists(key)` - Check if key exists
- `num()` - Count unique keys
- `foreach` - Iterate over keys

6. Advanced UVM Methods Reference (3/5)

- `do_copy(rhs)` - Copy configuration fields
- `convert2string()` - String representation
- Nested Configs - Config objects containing other configs
- `uvm_register_cb(Component, Callback)` - Register callback type
- `uvm_do_callbacks(Component, Callback, method(args))` - Execute callbacks
- `uvm_callbacks#(Component, Callback)::add(component, callback)` - Add callback

6. Advanced UVM Methods Reference (4/5)

- `uvm_callbacks#(Component, Callback)::delete(component, callback)` - Remove callback
- `convert2string()` - String representation
- `add_field(field)` - Add field to register
- `write(data)` - Write value to register
- `read()` - Read value from register
- `add_register(reg)` - Add register to block

6. Advanced UVM Methods Reference (5/5)

- `get_register(name)` - Look up register by name
- `write_register(name, data)` - Write register by name
- `read_register(name)` - Read register by name

Hands-on examples

Module 5 self-check

```
$ ./scripts/module5.sh --check
```

Exercise scaffold

```
./scripts/module5.sh --scaffold
```

Practice & assessment

Exercises

- Virtual Sequences
- Coverage Models
- Advanced Configuration
- Callbacks
- Register Model

Assessment checklist

- Can implement virtual sequences
- Can create coverage models
- Can use advanced configuration
- Can implement callbacks
- Can use register models
- Understands advanced UVM patterns

Summary & next steps

- Master advanced UVM features and patterns
- Complete module5/CHECKLIST.md
- Run ./scripts/module5.sh --check
- Next: Next module in course